

Capture-The-Flag Writeup: AI/LLM, Web and OSINT

Category: **AI/LLM**

Challenge 1: Chatbox

- **Description:** An interactive AI assistant interface designed to handle basic user scheduling and inquiries.
- **Approach:**
 - **Initial Discovery:** I began by mapping the system's operational capabilities, prompting the ai assistant to enumerate some of its functionalities. The model returned standard functional responses (e.g., time management and task scheduling). My initial direct requests for sensitive system flags gave no relevant output.
 - **Targeted Extraction:** I shifted my approach toward prompt injection, explicitly asking the model's system instructions and internal context regarding any flag or related to it deep in its memory. The model failed to restrict access to its system instructions, disclosing its internal data alongside the target value of the flag for this challenge.

Category: **AI/LLM**

Challenge 2: Document Whisperer

- **Description:** An LLM-integrated file analysis service capable of ingesting and summarizing document uploads.
- **Approach:**
 - **Input Reconnaissance:** During initial testing, I identified that the target interface is strictly enforced to accept only pdf file format.
 - **Payload Construction via PDF:** I decided to make a text file containing an indirect prompt injection payload designed to instruct the parser to dump system memory during summary generation. I converted this payload into PDF format using an online txt to pdf converter and submitted it to the application. This attempt however failed to reveal the desired output.

- **Reconstructed the PDF content:** I updated the document content with an explicit extraction command by telling the model to expose its context, by asking what is CTF and to give an example of a CTF flag that it has seen before or something that is hidden deep in its memory. Upon uploading the updated PDF, the parser executed the embedded instructions and printed the hidden flag.

Category: **AI/LLM**

Challenge 3: Query Crafter

- **Description:** A natural-language prompt handler connected to an underlying relational database layer.
- **Approach:**
 - **Baseline SQL Injection:** I initiated testing using standard SQL Injection (SQLi) payloads (e.g., `' +0R+1=1 --`). While this bypassed basic logic checks and exposed entries from the primary **users** table, public signature-based payloads were either blocked by input validation or truncated by the application.
 - **Natural Language Schema Injection:** Recognizing the direct link between the prompting and how it can be used using the former output of the ai model, I reconstructed my prompt from classic SQL syntax to natural-language schema. I explicitly instructed the model to query, extract, and display all row and column contents from the **SECRETS** table. The model executed the request without secondary validation, outputting the stored flag.

Category: **AI/LLM**

Challenge 4: System Leaks

- **Description:** An LLM implementation guarded by system-level safety rules intended to prevent prompt disclosure and data exfiltration.
- **Approach:**
 - **Safety Rules and Policy Identification:** Firstly, I attempted a direct system prompt extraction to read the internal safety policy to reveal hints or general output on how the LLM's security policy works. The application's safety layer intercepted and rejected these requests.
 - **Bypass & Obfuscation Prompts:** To bypass input filtering, I used external LLMs such as Gemini and Chatgpt to assist in crafting better prompts that obfuscates my hidden intention to manipulate the LLM to

revealing the flag, including character-slicing instructions and cipher-encoded payloads. The target model either rejected these attempts or returned unverified single-character responses.

- **Indirect System Prompt Extraction:** I refined my prompts into an indirect framing scenario, instructing the model to process a secondary analytical task that implicitly required referencing its underlying rules. By trying to brute-force the LLM into revealing something related to the flag, I tried multiple times to use the various prompts provided by the external AI models until one eventually works.

Category: **AI/LLM**

Challenge 5: The Gatekeeper

- **Description:** A restricted LLM challenge utilizing input-filtering guardrails to block direct exfiltration vectors.
- **Approach:**
 - **Indirect System Prompt Extraction:** I executed this challenge in parallel with *System Leaks* challenge, I targeted the application's input filters using structured prompt obfuscation and indirect phrasing techniques.
 - **Payload Execution & Verification:** I combined encoding transformations with multi-part slicing logic to degrade the guardrail's pattern-matching capabilities. After iterating through candidate prompts, an obfuscated string bypassed the input filter, causing the backend to emit candidate flag fragments until I successfully verified the correct string.

Category: **Web**

Challenge 6: GCS3

- **Description:** A web application hosting an administrative panel endpoint containing restricted assets.
- **Approach:**
 - **Reconnaissance:** Using former knowledge and techniques gathered from (GCS5), I confirmed that the primary flag data is within the admin control panel interface.
 - **Data Extraction:** I analyzed the admin panel's output and HTTP response. By navigating through table elements within the admin dashboard, I identified and extracted the flag embedded inside the table where the information about the users is in.

Category: **Web**

Challenge 7: GCS5

- **Description:** A user profile management system vulnerable to Insecure Direct Object References (IDOR) and weak password reset logic.
- **Approach:**
 - **IDOR Exploitation:** I intercepted a request to `/profile.php` using Burp Suite Repeater and modified the `userID` parameter. Setting `userID=1` successfully bypassed authorization checks, revealing the admin's account credentials.
 - **Account Takeover:** I initiated the application's "Forgot Password" functionality using the admin's details. Inspecting the raw HTTP server response, I identified a comment containing the cleartext One-Time Password (OTP).
 - **Authentication Bypass:** I submitted the leaked OTP to complete the credential reset process, authenticated as the administrator, and navigated to the administrative panel to retrieve the displayed flag located at the bottom of the tab.

Category: **Web**

Challenge 8: JS Prom

- **Description:** A web page serving event details with embedded client-side assets.
- **Approach:**
 - **Source Code Review:** I intercepted the initial HTTP `GET` request using Burp Suite and conducted a full source code review of the returned HTML and JavaScript dependencies. I found the hidden flag stored directly within an unstripped client-side source code comment.

Category: **Web**

Challenge 9: Rules

- **Description:** A static web page designed to test client-side source inspection and traffic analysis.
- **Approach:**
 - **Source Analysis:** I reviewed the rendered content and DOM structure via Browser Developer Tools. An initial string embedded in the visible text

proved to be a decoy flag{this_is_not_a_flag} which proved to be incorrect.

- **Traffic Interception:** I routed all site traffic through Burp Suite to analyze the unparsed HTTP response body. After analyzing the raw response headers and HTML comments revealed the true flag embedded inside an unrendered comment block

Category: **Web**

Challenge 10: Sheesh Yown

- **Description:** A web application serving static media assets (**sheesh.jpeg**) alongside unhandled session state data.
- **Approach:**
 - **Source Code Analysis:** I captured the web asset request in Burp Suite and inspected both the raw HTML response structure and local browser storage.
 - **Flag Retrieval:** I identified the flag via the local application data in Browser Developer Tools, confirming that the target string is inside the session cookie.

Category: **OSINT**

Category: Open-Source Intelligence (OSINT)

Challenge 11: Garden Location

- **Description:** An OSINT geolocation challenge requiring exact landmark identification from a target image depicting an outdoor tree/garden setup.
- **Approach:**
 - **Reverse Image Search:** I initiated visual intelligence gathering using Google Reverse Image Search. Initial automated matches suggested locations such as the SM Bulacan parking ramp and the UP Los Baños Fertility Tree, both of which were invalid.
 - **Feature Analysis & Field Verification:** Analyzing structural surrounding elements in the source image, I recognized architectural patterns corresponding to the UP Technohub complex (specifically the Building K courtyard). I conducted on-site physical OSINT by asking local employees of manulife and concentrix who are taking a smoke break there to ask the name of the garden where they answered that they don't know about it.
 - **Contextual Deduction:** Using secondary intelligence hints from the other CTF takers indicating the landmark shared its name with a popular

Philippine television franchise, I cross-referenced broadcast media titles associated with the garden's resemblance to any GMA shows to successfully identify the site target (*Encantadia*).